ATT 25ws

Exploring Latent Diffusion for Text Generation

Author: Heyuan Chi

March 30, 2026

This report studies latent diffusion for conditional text generation. We first test it on open-ended future continuation. In this setting, the autoencoder reconstructs well and the denoiser works well under oracle noise, but generation from Gaussian noise is still poor. This shows that a latent space can be good for reconstruction but still bad for generation. We then move to a stronger seq2seq task, QQP, and replace the original lightweight autoencoder with a BART-based latent autoencoder. In this new setting, free-run reconstruction becomes reliable, oracle denoising stays strong, and direct latent diffusion gives much more natural outputs. We also find that v-prediction is more stable than clean-latent prediction in long rollouts from pure noise. Overall, the results show that text latent diffusion depends a lot on the task, the latent space, and the diffusion objective.

Contents

1 Introduction **1**

2 Related Work **1**

3 Method **2**

3.1 Problem formulation and notation 2
3.2 Stage I: open-ended future continuation 2
3.3 Conditional latent diffusion 2
3.4 Diffusion objectives 3
3.5 Reverse diffusion and sampling 3
3.6 Stage II: seq2seq BART latent diffusion 4
3.7 Diagnostic framework 4

4 Experiments **4**

4.1 Experimental setup 4
4.2 Open-ended future continuation 4
4.3 QQP with BART latents 5
4.4 Direct latent diffusion on QQP 6

5 Conclusion **6**

1 Introduction

Most text generation systems still use token-level autoregressive decoding [1, 2]. Another option is to generate in a continuous latent space and then decode the latent back into text [3, 4]. Diffusion models make this idea attractive because they generate by iterative denoising [5]. Recent work has shown that this can work for controllable generation, seq2seq generation, and latent language modeling [6, 7, 8, 9]. Still, text diffusion is very sensitive to the latent space and to the task.

There are good reasons to study this direction. A latent model may give a more global view of the target text. It may also avoid strict left-to-right generation in token space. But these advantages only matter if the latent space is both easy to decode and easy to generate. This is the main point we want to test.

This paper studies that problem with experiments. We start from open-ended future continuation. The model must generate the latent of a future text block from a prefix. In this setting, the autoencoder reconstructs well and the denoiser works well under oracle corruption. But generation from a Gaussian prior fails. This result is important because it shows that a latent space can be good for reconstruction and local denoising, but still bad for conditional generation from noise.

In the second part, we move to QQP paraphrase generation [10]. We also replace the original lightweight autoencoder with a BART-based latent autoencoder [11]. This setup is closer to recent latent diffusion work for language [9]. In this new setting, free-run reconstruction becomes reliable, direct latent diffusion gives much more fluent text, and v-prediction is more stable than clean-latent prediction in long pure-noise rollouts. The main point of this paper is simple: latent diffusion for text depends a lot on task openness, latent design, and diffusion objective.

2 Related Work

Work on latent text generation is older than diffusion. Sentence-level and hierarchical latent-variable models already showed that a continuous latent can capture structure beyond local next-token prediction [3, 4]. They also showed an important problem: a latent can be useful but still hard to sample from.

Recent diffusion work studies this issue in several ways. Diffusion-LM studies controllable generation in continuous space [6]. DiffuSeq and SeqDiffuSeq show that diffusion can work in strongly conditioned seq2seq tasks, and they also show that conditioning and training details matter a lot [7, 8]. D3PM studies diffusion in discrete state spaces [12]. The closest paper to our final setup is Latent Diffusion for Language Generation [9]. That work trains diffusion directly on the latent space of a pretrained encoder-decoder model. Our second-stage system follows the same general direction, but this paper puts more focus on diagnosis. We use the shift from open-ended continuation to seq2seq BART latents to show what changes when latent diffusion becomes easier to train and sample from.

Pretrained sequence-to-sequence models are also important for this topic. BART is trained as a denoising encoder-decoder model and already has a strong decoder [11]. This makes it a natural base for a latent autoencoder. In our work, this is not only a strong baseline. It is also a way to test whether a better language prior can make the latent space easier to use for diffusion.

3 Method

3.1 Problem formulation and notation

We study conditional text generation with input context c and target text $y = (y_1, \ldots, y_T)$. We do not model $p(y \mid c)$ directly in token space. Instead, we first encode y into a latent z_0, and then learn a model for $p(z_0 \mid c)$.

Let E_ϕ denote the encoder and D_ψ the decoder. The latent representation is

$$z_0 = E_\phi(y), \tag{1}$$

where $z_0 \in \mathbb{R}^{L \times d}$ is a sequence of L latent slots of dimension d. The decoder then reconstructs the target text autoregressively from the latent:

$$p_\psi(y \mid z_0) = \prod_{t=1}^{T} p_\psi(y_t \mid y_{<t}, z_0). \tag{2}$$

The autoencoder is trained with token-level negative log-likelihood:

$$\mathcal{L}_{\mathrm{AE}}(\phi, \psi) = -\log p_\psi(y \mid E_\phi(y)). \tag{3}$$

3.2 Stage I: open-ended future continuation

In the first stage, c is a prefix text block and y is a future text block. The autoencoder maps the future block into a compact latent with a small number of slots. In the best version, the latent is compressed into four slots and then refined with a causal residual refinement module.

Let $H \in \mathbb{R}^{T \times h}$ be the token-level hidden states produced by the future encoder. The compact latent is formed by cross-attending a learned set of slot queries to H:

$$\tilde{Z} = \mathrm{CrossAttn}(Q_{\mathrm{slot}}, H, H), \tag{4}$$

and we then apply optional causal slot refinement:

$$Z = \tilde{Z} + \lambda\, \mathrm{CausalRefine}(\tilde{Z}), \tag{5}$$

where λ is a residual scaling factor. The latent Z is then expanded and decoded back to the target block.

On top of this latent space, we study two generation strategies. The first is a direct predictor $P_\eta(c)$ that regresses to the target latent. The second is a latent denoiser trained with a diffusion objective.

3.3 Conditional latent diffusion

For both stages, the diffusion component follows the standard forward noising process of DDPMs [5], applied in latent space. Given a clean latent z_0, a timestep $t \in \{0, \ldots, T_{\mathrm{diff}} - 1\}$, and Gaussian noise $\epsilon \sim \mathcal{N}(0, I)$, we construct

$$z_t = \sqrt{\bar{\alpha}_t}\, z_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \tag{6}$$

where $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$ and $\alpha_s = 1 - \beta_s$.

The denoiser f_θ takes the noisy latent z_t, a context representation h_c, and a timestep embedding:

$$\hat{u}_t = f_\theta(z_t, h_c, t, \hat{z}^{\text{sc}}), \tag{7}$$

where $\hat{z}^{\text{sc}}$ is an optional self-conditioning signal. In our implementation, self-conditioning uses a detached prediction from a previous denoising pass with probability 0.5.

The context representation is produced by a separate context encoder:

$$h_c = g_\gamma(c). \tag{8}$$

For open-ended continuation, c is the prefix block. For seq2seq generation, c is the source sequence.

3.4 Diffusion objectives

We study two denoising targets.

Clean-latent prediction. Under this objective, the model directly predicts $\hat{z}_0$ from z_t:

$$\hat{z}_0 = f_\theta(z_t, h_c, t, \hat{z}^{\text{sc}}), \tag{9}$$

and the training objective is

$$\mathcal{L}_{x_0}(\theta) = \mathbb{E}_{z_0,\epsilon,t}\left[\|\hat{z}_0 - z_0\|_2^2\right]. \tag{10}$$

v-prediction. We also study v-prediction. We define the target

$$v_t = \sqrt{\bar{\alpha}_t}\,\epsilon - \sqrt{1-\bar{\alpha}_t}\,z_0. \tag{11}$$

The denoiser predicts $\hat{v}_t$, and the training objective becomes

$$\mathcal{L}_v(\theta) = \mathbb{E}_{z_0,\epsilon,t}\left[\|\hat{v}_t - v_t\|_2^2\right]. \tag{12}$$

At inference time, $\hat{v}_t$ is converted back to a clean-latent estimate by

$$\hat{z}_0 = \sqrt{\bar{\alpha}_t}\,z_t - \sqrt{1-\bar{\alpha}_t}\,\hat{v}_t. \tag{13}$$

3.5 Reverse diffusion and sampling

During sampling, we use deterministic DDPM-style mean updates. When a clean latent estimate $\hat{z}_0$ is available, we first recover the implied noise estimate

$$\hat{\epsilon}_t = \frac{z_t - \sqrt{\bar{\alpha}_t}\,\hat{z}_0}{\sqrt{1-\bar{\alpha}_t}}. \tag{14}$$

We then use the DDPM posterior mean update [5]:

$$z_{t-1} = \frac{z_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\hat{\epsilon}_t}{\sqrt{\alpha_t}}. \tag{15}$$

In the current implementation, sampling is deterministic. We use the posterior mean and do not add extra Gaussian noise. This makes rollouts easier to compare.

3.6 Stage II: seq2seq BART latent diffusion

In the second stage, we move to QQP paraphrase generation and replace the original lightweight autoencoder with a BART-based latent autoencoder [11, 9]. Let H_{enc} be the hidden states of the pretrained BART encoder. We compress them into a latent bottleneck using a Perceiver-style resampler:

$$z_0 = \text{PerceiverEnc}(H_{\text{enc}}). \tag{16}$$

The latent is then decoded back into a sequence of decoder-side hidden states,

$$\tilde{H}_{\text{dec}} = \text{PerceiverDec}(z_0), \tag{17}$$

which are fed into the pretrained BART decoder. The target text is reconstructed with standard teacher-forced sequence-to-sequence likelihood:

$$\mathcal{L}_{\text{BART-AE}} = -\log p_\psi(y \mid \tilde{H}_{\text{dec}}). \tag{18}$$

This design differs from the first-stage autoencoder in two ways. It uses a pretrained decoder, which gives a stronger language prior. It also puts the bottleneck on top of pretrained encoder states instead of learning the whole latent space from scratch.

After building the BART latent autoencoder, we train direct latent diffusion on top of it. Unlike the earlier predictor-refinement experiments, the seq2seq system directly learns a conditional denoiser from source question to target latent.

3.7 Diagnostic framework

We evaluate the system at multiple levels, not only by final text quality. We look at autoencoder reconstruction, oracle denoising from noisy target-centered latents, pure-noise generation from the prior, predictor-initialized refinement, and the gap between teacher-forced decoding and free-run decoding. This helps us separate local denoising success from true conditional generation success.

4 Experiments

4.1 Experimental setup

The experiments are divided into two stages. Stage I studies open-ended future continuation on prefix/future splits derived from WikiText-style data. Stage II studies QQP paraphrase generation as a strongly conditioned seq2seq problem. In both stages we report validation losses, and we also track latent mean-squared error (MSE) on fixed diagnostic examples. This helps us separate reconstruction, denoising from noisy target-centered latents, and generation from a Gaussian prior.

4.2 Open-ended future continuation

The first stage shows a clear pattern. The best future-block autoencoder uses four latent slots with causal residual refinement and reaches a validation loss of about 0.2600, but downstream generation is still weak. Moving from a deterministic predictor to a CVAE improves the latent

System	Metric	Value
4-slot AE + causal residual refinement	best val. loss	0.25997
Deterministic predictor	latent MSE	1.6039
CVAE predictor	latent MSE	1.3727
Prefix-conditioned denoiser	oracle latent MSE	0.2870
Prefix-conditioned denoiser	pure-noise latent MSE	2.0413
Joint predictor–denoiser	predictor-init latent MSE	1.3389
Joint predictor–denoiser	refined latent MSE	1.4432

Table 1: Stage I results on open-ended future continuation. Autoencoder numbers are best validation losses; generation metrics are measured on a fixed diagnostic sample that was used throughout model development.

Initialization	Representative Start Noise	Latent MSE to Target
Pure Gaussian start + 50-step rollout	full prior	3.0848
Target latent + noise, reverse from $t = 5$	low noise	0.0270
Target latent + noise, reverse from $t = 15$	medium-low noise	0.0795
Target latent + noise, reverse from $t = 30$	medium-high noise	0.1723
Target latent + noise, reverse from $t = 49$	highest training noise	0.3110

Table 2: Representative denoiser diagnostic on the open-ended future-continuation setup. The model can recover the target latent from highly noisy target-centered states, but not from an unconditional Gaussian start.

MSE only a little. The denoiser behaves much better under oracle corruption than under prior sampling. This already suggests that local recovery and full generation are not the same problem.

To localize the failure, we compare rollouts from pure Gaussian noise with rollouts that start from noisy versions of the true target latent. Reverse diffusion succeeds in the second case, even at high noise levels, but fails in the first. Table 2 shows a representative example.

This makes the failure mode fairly clear. The denoiser learns a useful local reverse process, but it does not reliably enter the target region from the prior. Predictor-initialized refinement and joint predictor–denoiser training do not change this result. Good reconstruction and good oracle denoising are not enough to make the latent space generation-friendly.

4.3 QQP with BART latents

The second stage tests two ideas from Stage I: the task may be too open-ended, and the latent may be too specialized for reconstruction. We therefore move to QQP paraphrase generation and replace the lightweight autoencoder with a BART-based latent autoencoder. In this setting, free-run reconstruction becomes reliable. Table 3 shows the transition from the original latent to the BART latent.

This matters because poor downstream generation can no longer be blamed on a weak decoder.

AE setting	Train loss	Val loss
4-slot causal-refine AE (future continuation)	—	0.25997
BART latent AE, future continuation (best epoch ≈ 29)	—	0.31986
QQP BART latent AE (epoch 17)	0.0871	0.0304

Table 3: Autoencoder results before and after the pivot to a pretrained BART latent space. Free-run reconstruction on QQP is correct on the representative validation sample used in the diffusion diagnostics.

Setting	Train loss	Val loss	Oracle MSE	Prior MSE
QQP direct diffusion (x_0), epoch 28	0.0400	0.0354	0.0607	1.4056 at 25 steps
QQP direct diffusion (x_0), epoch 28	0.0400	0.0354	0.0607	1.5791 at 50 steps
QQP direct diffusion (v), early checkpoint	0.8541	0.7951	0.1351	1.0991 at 50 steps

Table 4: Representative QQP direct-diffusion results after the pivot to the BART latent space. The v-prediction numbers are preliminary but already indicate lower error in the long-rollout regime.

4.4 Direct latent diffusion on QQP

Once the BART latent autoencoder is ready, we train direct latent diffusion conditioned on the source question. The denoiser training curves are smooth under both clean-latent prediction and v-prediction. Under the clean-latent objective, the validation loss drops from 0.2509 at epoch 1 to about 0.0354 by epoch 28. Oracle denoising is also strong, with representative latent MSE around 0.0607.

Pure-noise sampling still does not fully recover the target paraphrase, but the outputs are much more grammatical than in Stage I. So the system moves into a narrower failure mode: it learns a useful language prior in latent space, but conditional alignment is still incomplete. We then replace clean-latent prediction with v-prediction. The early results suggest that this improves long pure-noise rollouts, with lower 50-step latent MSE on the representative probe. Table 4 summarizes the comparison.

The QQP experiments therefore look different from the first-stage experiments. The model no longer fails because the latent cannot be decoded or because the output is obviously broken. The remaining problem is conditional alignment.

5 Conclusion

This paper studies latent diffusion for text generation through diagnostic experiments. The first-stage results on open-ended future continuation show that strong reconstruction and strong oracle denoising do not imply successful generation from pure noise. The latent space learned there is useful for reconstruction and local recovery, but not good enough for conditional rollout from the prior.

This diagnosis leads to a move to QQP paraphrase generation and to a pretrained BART-based

latent autoencoder. In that setting, free-run reconstruction becomes reliable, oracle denoising remains strong, and direct latent diffusion produces fluent question-like outputs. The remaining problem is not collapse, but imperfect conditional alignment.

The final result is that text latent diffusion depends strongly on task openness, latent design, and diffusion objective. Once the task and latent become better matched, changes in the diffusion recipe begin to matter in the expected way. In particular, v-prediction improves pure-noise rollout stability. The most promising next step is stronger conditional control in the QQP+BART setting.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *arXiv preprint arXiv:1706.03762*, 2017.

[2] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[3] Samuel R. Bowman, Luke Vilnis, Oriol Vinyals, Andrew M. Dai, Rafal Jozefowicz, and Samy Bengio. Generating sentences from a continuous space. *arXiv preprint arXiv:1511.06349*, 2016.

[4] Seonghan Park, Dong Bok Kim, Xiao Lu, and Sungroh Cho. A hierarchical latent structure for variational conversation modeling. *arXiv preprint arXiv:1804.03424*, 2018.

[5] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *arXiv preprint arXiv:2006.11239*, 2020.

[6] Xiang Lisa Li, John Thickstun, Ishaan Gulrajani, Percy Liang, and Tatsunori B. Hashimoto. Diffusion-lm improves controllable text generation. *arXiv preprint arXiv:2205.14217*, 2022.

[7] Shansan Gong, Mukai Li, Jiangtao Feng, Zhiyong Wu, and Lingpeng Kong. Diffuseq: Sequence to sequence text generation with diffusion models. *arXiv preprint arXiv:2210.08933*, 2023.

[8] Hongyi Yuan, Zheng Yuan, Chuanqi Tan, Fei Huang, and Songfang Huang. Seqdiffuseq: Text diffusion with encoder-decoder transformers. *arXiv preprint arXiv:2212.10325*, 2023.

[9] Justin Lovelace, Varsha Kishore, Chao Wan, Eliot Shekhtman, and Kilian Q. Weinberger. Latent diffusion for language generation. *Advances in Neural Information Processing Systems*, 2023.

[10] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461*, 2018.

[11] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461*, 2020.

[12] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. *arXiv preprint arXiv:2107.03006*, 2021.